

# Vue 3 Dasar

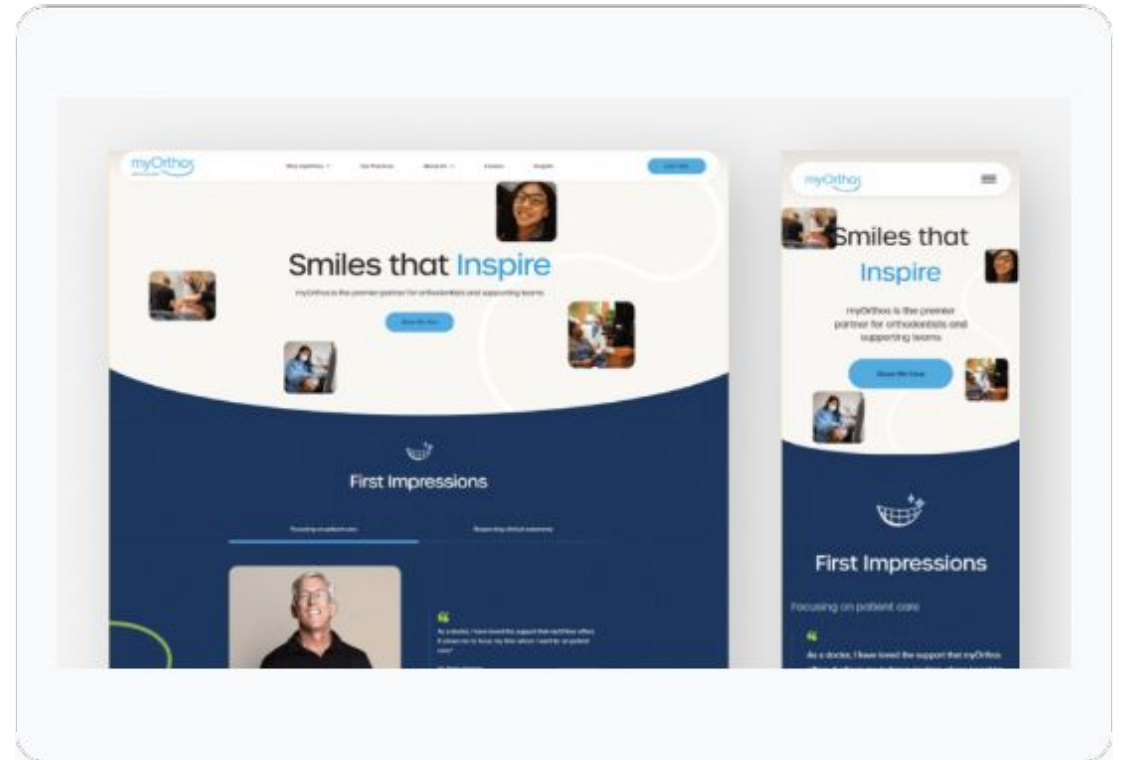
Event Handling, Conditional Rendering, List Rendering, dan Form  
Binding

# Prasyarat Pembelajaran

## Fondasi yang Dibutuhkan

Sebelum memasuki materi ini, peserta diharapkan telah memahami konsep dasar Vue 3 yang meliputi Vite, penggunaan script setup, template syntax, pembuatan component, pengiriman props, serta reaktivitas menggunakan `ref()` dan `reactive()`.

Keempat materi dalam presentasi ini berfokus pada pembangunan interaksi pengguna di atas reactive state yang telah dipelajari sebelumnya.



# Tujuan Pembelajaran



**Event Handling:** Menangani event seperti click, input, dan submit beserta modifikatornya.



**Conditional Rendering:** Menampilkan atau menyembunyikan elemen menggunakan v-if, v-else-if, v-else, dan v-show.

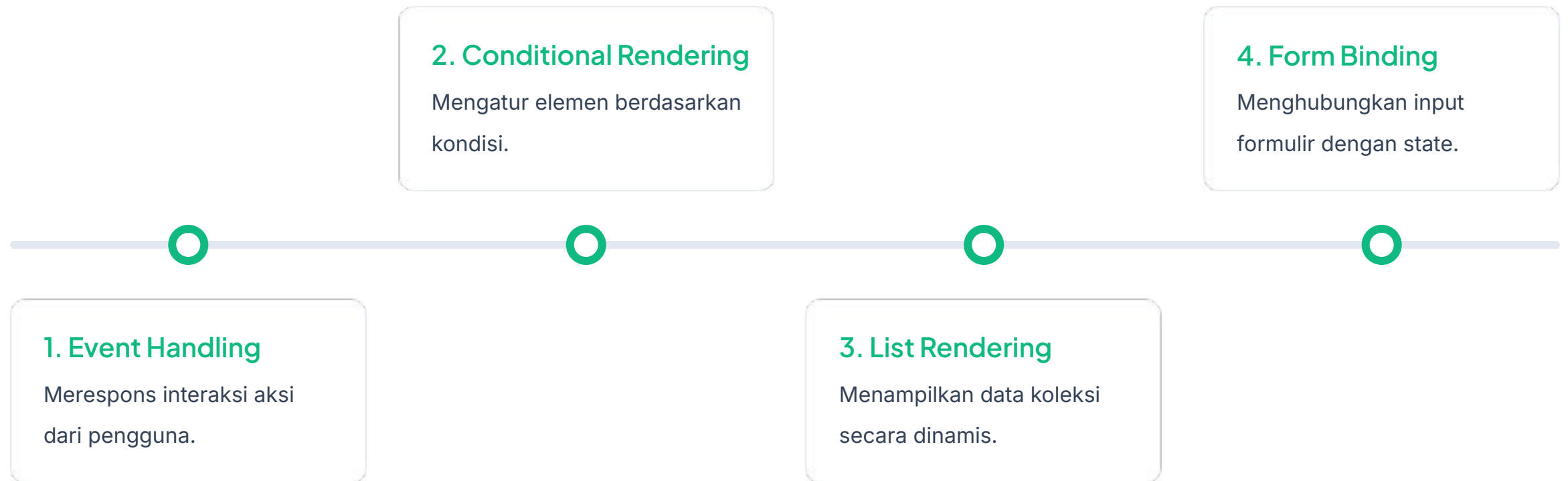


**List Rendering:** Menampilkan koleksi data berulang menggunakan v-for dan memahami peran atribut key.



**Form Binding:** Menghubungkan elemen input formulir dengan state reaktif menggunakan v-model.

# Roadmap Pembelajaran



# Dari State ke UI Interaktif

## Siklus Interaktivitas Vue

Dalam Vue, antarmuka pengguna bereaksi secara langsung terhadap interaksi. Pola kerjanya sangat sederhana: Pengguna melakukan aksi, aksi memicu event, event memperbarui state, dan Vue merender ulang UI yang terdampak secara otomatis.

## Hasil Akhir

Dengan siklus ini, antarmuka pengguna kemudian dapat muncul atau menghilang secara dinamis, menampilkan koleksi data dari server, dan menerima input formulir dengan sangat mulus.



# Event Handling

Cara Vue merespons aksi dari pengguna secara langsung

# Apa Itu Event Handling?

# Event

**Aksi Pengguna**

## Merespons Tindakan

Event Handling adalah cara sistem merespons aksi spesifik dari pengguna. Aksi ini bisa berupa klik tombol, pengetikan teks pada kolom input, pengiriman formulir, atau sekadar pergerakan kursor (hover) di atas sebuah elemen HTML. Untuk menangani event tersebut, Vue menyediakan direktif khusus yaitu v-on atau versi singkatnya menggunakan simbol @.

# Direktif v-on dan Simbol @

## Penulisan Lengkap (v-on)

Penulisan formal menggunakan direktif utuh bawaan dari Vue.

```
<button v-on:click="increment">  
  Tambah  
</button>
```

## Shorthand (Simbol @)

Keduanya benar-benar ekuivalen. Dalam praktik pengembangan sehari-hari, versi shorthand lebih disukai karena lebih ringkas.

```
<button @click="increment">  
  Tambah  
</button>
```



# Daftar Event Umum

| Nama Event | Keterangan Aksi                                                 |
|------------|-----------------------------------------------------------------|
| @click     | Dijalankan saat pengguna melakukan klik kiri pada elemen.       |
| @input     | Dijalankan setiap kali ada perubahan pada nilai kolom input.    |
| @submit    | Dijalankan saat formulir dikirim atau tombol submit ditekan.    |
| @mouseover | Dijalankan saat kursor (pointer) bergerak masuk ke area elemen. |

# Implementasi Event Click

Mendeklarasikan fungsi dalam script setup, kemudian menautkannya pada elemen tombol HTML menggunakan direktif @click.

```
<script setup>
import { ref } from "vue"
const count = ref(0)

const increment = () => {
  count.value++
}
</script>
```

```
<template>
  <p>Count: {{ count }}</p>
  <button @click="increment">
    Tambah
  </button>
</template>
```

# Alur Logika Event Click



## 1. User Klik

Pengguna mengklik elemen UI yang memicu penangkapan oleh @click.



## 2. Handler Jalan

Fungsi increment() dieksekusi, lalu memanipulasi reactive state (count.value++).



## 3. Render Ulang

Vue mendeteksi mutasi data dan secara cerdas memperbarui blok antarmuka terkait.

# Function Handler vs Inline Expression

## Function Handler

Logika ditulis di dalam sebuah fungsi di script, lalu dipanggil di template.

```
<button @click="increment">  
  Tambah  
</button>
```

Sangat cocok untuk blok logika sistem yang kompleks atau membutuhkan banyak pemrosesan.

## Inline Expression

Logika sederhana dieksekusi langsung di dalam penulisan tag template.

```
<button @click="count++">  
  Tambah  
</button>
```

Lebih ringkas dan cocok untuk pengoperasian logika matematis sederhana.

# Event Input dan Object Event

## Menangkap Nilai Input

Event input memberikan sebuah objek event secara otomatis. Dari objek ini, kita dapat mengakses elemen pemicu melalui **e.target** dan mendapatkan nilai akhirnya di **e.target.value**.

```
<input
  @input="e => name = e.target.value"
  placeholder="Ketik nama..."
/>
<p>Halo, {{ name }}</p>
```



# Penanganan Event Submit

## Mengirim Formulir

Event @submit ditempelkan pada elemen form. Ketika pengguna menekan enter atau tombol bertipe submit, fungsi yang ditautkan akan dijalankan untuk memproses data.

## Mencegah Reload Browser

Secara default, browser akan me-reload halaman penuh saat form dikirim. Untuk mencegah sifat asli ini di Vue, kita menambahkan modifier .prevent.

```
<form @submit.prevent="handleSubmit">  
  <!-- input fields -->  
  <button type="submit">Kirim</button>  
</form>
```

# Ringkasan Event Modifier Vue

| Modifier        | Fungsi Utama                                             | Contoh Penulisan |
|-----------------|----------------------------------------------------------|------------------|
| <b>.prevent</b> | Mencegah perilaku standar peramban web.                  | @submit.prevent  |
| <b>.stop</b>    | Menghentikan perambatan (bubbling) dari child ke parent. | @click.stop      |
| <b>.once</b>    | Memastikan bahwa handler dieksekusi maksimal satu kali.  | @click.once      |



# Conditional Rendering

Membentuk antarmuka pintar yang tampil hanya saat kondisi terpenuhi



# Apa Itu Conditional Rendering?



## Konsep Dasar

Conditional rendering adalah metode untuk mengatur apakah sebuah elemen HTML akan dirender (ditampilkan) di browser atau tidak, murni berdasarkan evaluasi kondisi boolean tertentu.



## Direktif Pendukung

Untuk mencapai hal ini, Vue menyediakan serangkaian direktif percabangan standar meliputi v-if, v-else-if, v-else, serta alternatif yang sedikit berbeda secara mekanisme yaitu v-show.

# Penggunaan v-if dan v-else

## Percabangan Dasar

Direktif v-if merender elemen hanya jika kondisinya mutlak benar. Direktif v-else digunakan tepat di bawah elemen tersebut sebagai alternatif otomatis jika kondisi bernilai salah.

```
<p v-if="isLoggedIn">
  Selamat datang kembali!
</p>
<p v-else>
  Silakan login terlebih dahulu.
</p>
```

Pastikan posisi tag v-else harus segera diletakkan berdampingan tepat setelah blok elemen tag v-if, agar sistem perenderan Vue dapat memahaminya sebagai satu kesatuan rute yang valid.

# Pengkondisian Rantai: v-else-if

# Cek

## Rantai Kondisi

### Struktur Bertingkat

Vue mengevaluasi kondisi secara berurutan dari atas menuju ke bawah dan langsung memilih cabang logika yang pertama kali memenuhi syarat kebenaran sistem.

```
<p v-if="score >= 90">Nilai A</p>  
<p v-else-if="score >= 75">Nilai B</p>  
<p v-else-if="score >= 60">Nilai C</p>  
<p v-else>Nilai D</p>
```

# Alternatif Tampilan: v-show

## Menyembunyikan Tampilan

Direktif v-show secara fungsional terlihat sama seperti v-if, yaitu mengontrol visibilitas elemen. Namun cara kerjanya di belakang layar memiliki perbedaan fundamental yang signifikan.

## Manipulasi CSS

Elemen dengan v-show selalu dirender dan tetap ada di dalam struktur DOM HTML seutuhnya. Vue hanya mengontrol atribut CSS display (menjadi none atau block) untuk mengatur penampakannya.

```
<p v-show="isVisible">  
  Teks ini bisa disembunyikan  
</p>
```

# Perbandingan Fundamental v-if vs v-show

| Aspek Evaluasi             | Karakter v-if                                             | Karakter v-show                                               |
|----------------------------|-----------------------------------------------------------|---------------------------------------------------------------|
| Intervensi DOM             | Elemen dihapus / ditambahkan kembali ke DOM secara fisik. | Elemen tetap hidup secara penuh di DOM.                       |
| Biaya Toggle (Bolak-balik) | Kinerja lebih berat jika kondisi ditukar terlalu sering.  | Sangat ringan dan cepat karena sekadar mengubah properti CSS. |

# Strategi Penggunaan Terbaik



## Gunakan v-if

Ketika kondisi tidak terlalu sering berubah atau status awal elemen bernilai salah sehingga komponen kompleks tersebut lebih baik diabaikan dan tidak dirender sama sekali untuk efisiensi.



## Gunakan v-show

Ketika sebuah elemen akan sangat sering dan berulang kali dibuka dan ditutup (toggle). Contoh terbaik adalah komponen menu dropdown, panel notifikasi, atau kotak dialog modal statis.



# List Rendering

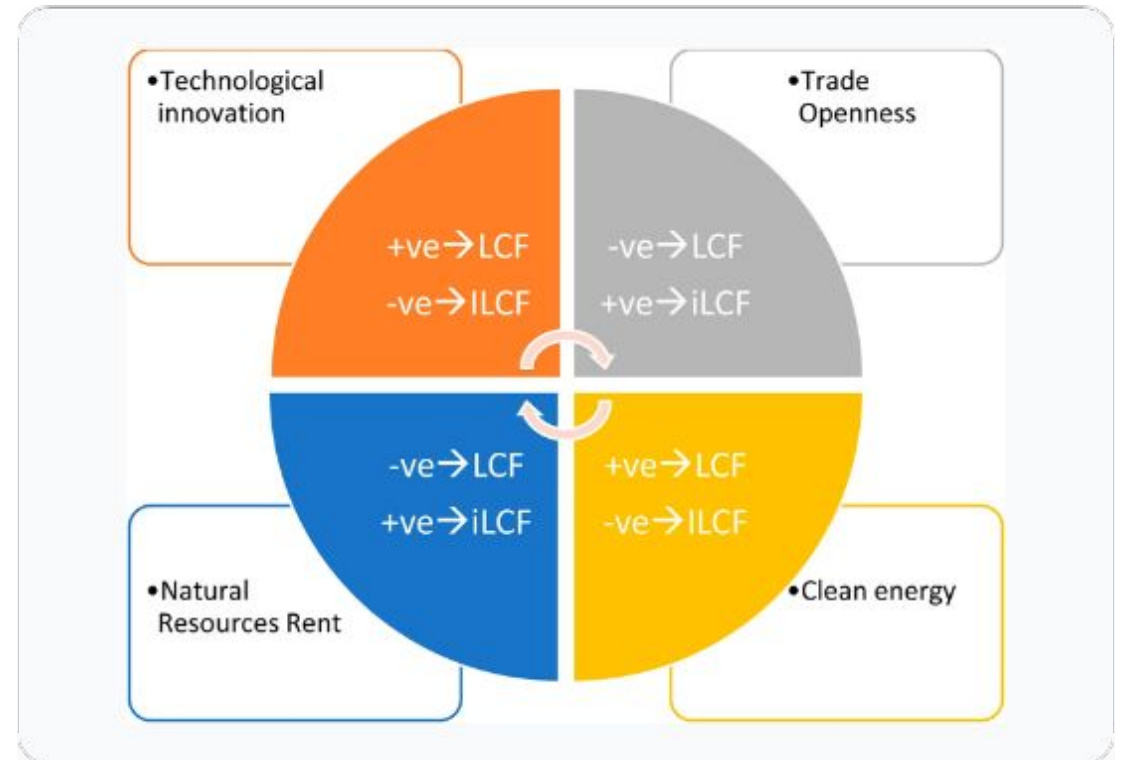
Mencetak serangkaian elemen HTML berulang berdasarkan koleksi data

# Apa Itu List Rendering?

## Iterasi Data Otomatis

List rendering adalah teknik ampuh untuk menampilkan data secara terstruktur dalam bentuk daftar berulang secara instan, menggunakan sumber data utama yang bertipe array atau objek terstruktur.

Fungsi repetisi otomatis ini dikendalikan oleh direktif andalan Vue yang bernama v-for, memberikan efisiensi luar biasa dalam pengkodean tanpa batas.





# Perulangan Array Sederhana

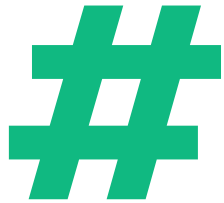
## Struktur v-for Dasar

Sintaks v-for menuntut pola penulisan yang spesifik, yaitu `item in items`, di mana `items` merepresentasikan data array induk, sedangkan `item` mewakili satu buah elemen di putaran perulangan saat itu.

## Contoh Kode

```
<ul>
  <li v-for="fruit in fruits" :key="fruit">
    {{ fruit }}
  </li>
</ul>
```

# Memanfaatkan Nomor Index



## Urutan Dinamis

### Mengakses Posisi Elemen

Vue mengizinkan ekstraksi parameter kedua dari perulangan v-for untuk mendapatkan posisi indeks elemen yang bermula dari angka nol mutlak.

```
<li v-for="(fruit, index) in fruits" :key="index">
  {{ index + 1 }}. {{ fruit }}
</li>
```

Untuk menampilkan nomor urut di layar secara wajar, operasikan matematika  $\text{index} + 1$  secara langsung di dalam interpolasi.

# Peran Krusial Atribut Key



**Penanda Identitas:** Atribut key memberikan identitas unik yang absolut bagi setiap perulangan item list Vue.



**Integritas DOM:** Membantu mesin rendering Vue melacak elemen secara akurat untuk meminimalisasi kesalahan pembaharuan posisi dan menjaga integritas state elemen asli.



**Best Practice Data:** Selalu prioritaskan penggunaan ID stabil milik data (misal user.id) ketimbang posisi index array semata yang berisiko bergeser sewaktu-waktu.

# Render Array Bersarang Objek

## Struktur Koleksi Data Asli

Dalam realitas pengembangan, data mayoritas disajikan dalam bentuk himpunan objek (rekaman) yang memuat aneka properti kompleks dan ID yang valid.

```
const users = [  
  { id: 1, name: "Budi" },  
  { id: 2, name: "Siti" }  
]
```

## Penerapan di Template

Gunakan format notasi titik secara presisi untuk menembus dan merender detail spesifik properti milik objek tersebut di antarmuka.

```
<li v-for="user in users" :key="user.id">  
  {{ user.name }}  
</li>
```

# Nested Loop (Perulangan Ganda)

## Perulangan di Dalam Perulangan

Data terstruktur yang berjenjang (seperti daftar kategori produk yang memiliki sub-produk turunan) menuntut kombinasi perulangan v-for di luar yang melingkupi perulangan v-for lapisan kedua di dalamnya.

```
<div v-for="category in categories"
      :key="category.name">
  <h4>{{ category.name }}</h4>
  <ul>
    <li v-for="item in category.items"
          :key="item">
      {{ item }}
    </li>
  </ul>
</div>
```

# v-for + Component + Props

## Menggandakan Komponen UI

Bukan hanya murni elemen HTML standar, direktif canggih v-for sangat kompatibel apabila digunakan untuk menggandakan instansi blok Single File Component kustom buatan Anda sendiri.

## Integrasi Binding Props

Lewati data setiap hasil perputaran elemen langsung menuju dalam komponen menggunakan sistem props dengan mulus.

```
<UserCard  
  v-for="user in users"  
  :key="user.id"  
  :name="user.name"  
>
```



# Form Binding

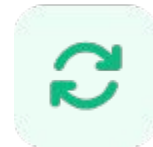
Manajemen koneksi dua arah antara isian input dan penyimpanan  
state

# Memahami Two-Way Binding



## Konsep Dua Arah

Form Binding mengikatkan elemen input formulir dengan wadah state milik Vue secara dua arah menggunakan direktif v-model.



## Sinkronisasi Instan

Setiap ketikan user akan seketika tersimpan di state. Sebaliknya, mutasi langsung dari logika sistem akan refleksi merubah visual input form.



# Input Text dan Area Pesan

## Input Standard

Menangkap data masukan teks satu baris secara normal.

```
<input
  v-model="name"
  placeholder="Ketik nama..."
/>
<p>Halo, {{ name }}</p>
```

## Textarea Multi Baris

Tangkapan masukan teks berupa paragraf panjang layaknya isian blok pesan lengkap.

```
<textarea
  v-model="message"
  placeholder="Tulis pesan...">
</textarea>
```

# Penanganan Pilihan Tunggal & Jamak

## Kotak Centang (Checkbox)

Bisa mengembalikan data kebenaran boolean (true/false) mutlak, atau dikompilasi secara agregat sebagai array jamak dari beragam pilihan berbeda.

```
<input type="checkbox" v-model="agree" />  
<p>Status Persetujuan: {{ agree }}</p>
```

## Tombol Radio Sederhana

Memaksa user memilih eksklusif hanya pada satu nilai unik dalam kelompok opsi tersebut.

```
<input type="radio" value="L" v-model="gender" />  
<input type="radio" value="P" v-model="gender" />
```

# Manajemen Select Dropdown

## List

**Opsi Terpilih**

### Pengikatan Kotak Opsi

Direktif v-model ditanamkan tepat di elemen select induk, bukan di anaknya. Secara teknis ia akan melacak dan menarik value statis milik opsi yang sukses dipilih user.

```
<select v-model="city">  
  <option disabled value="">Pilih Kota</option>  
  <option>Jakarta</option>  
  <option>Bandung</option>  
</select>
```

# Deretan Modifier V-Model Spesial

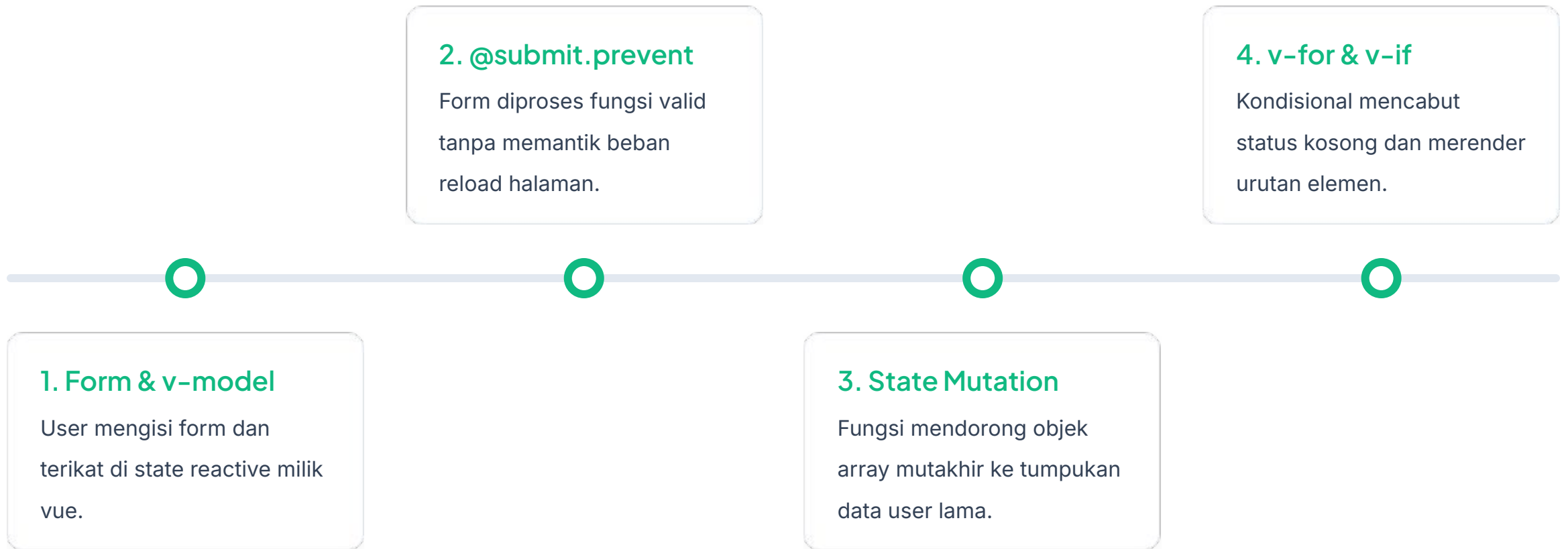
| Tipe Modifikasi | Keterangan dan Kegunaan                                                                                                      |
|-----------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>.lazy</b>    | Menunda proses pembaruan state sinkron; sistem menunggu event change/blur ketimbang saat user sibuk mengetik beruntun.       |
| <b>.trim</b>    | Langsung menyingkirkan atau membersihkan kotoran karakter spasi tak terlihat di awal dan ujung akhir masukan.                |
| <b>.number</b>  | Menginstruksikan sistem melakukan usaha konversi nilai string secara transparan menjadi bilangan angka sejati di Javascript. |



# Kesatuan Harmoni Vue

Event Handling + Conditional Rendering + List Rendering + Form  
Binding + Reactive State berpadu membidani interaktivitas sempurna  
pada aplikasi berbasis web

# Arsitektur Alur Sistem Latihan



# Analisis Kesalahan Umum Developer



**Lupa Membuka `.value`:** Mengakses manipulasi state bentukan fungsi `ref()` di javascript harus melampirkan panggilan suffix `.value`.



**Kecolongan Reload:** Lupa memberikan pembatas sakti `.prevent` pada form yang seketika menghapus sesi halaman tanpa henti.



**Pengabaian Key Valid:** V-for digunakan polos tanpa penjamin `:key` solid; berisiko menyebabkan bug visual yang sulit dilacak.



**Pemahaman Salah v-show:** Terjebak mitos v-show menghilangkan data dari elemen pohon; v-show nyatanya sekadar main CSS display belaka.

# Uji Kompetensi Konsep Interaksi

## Pertanyaan Terbatas

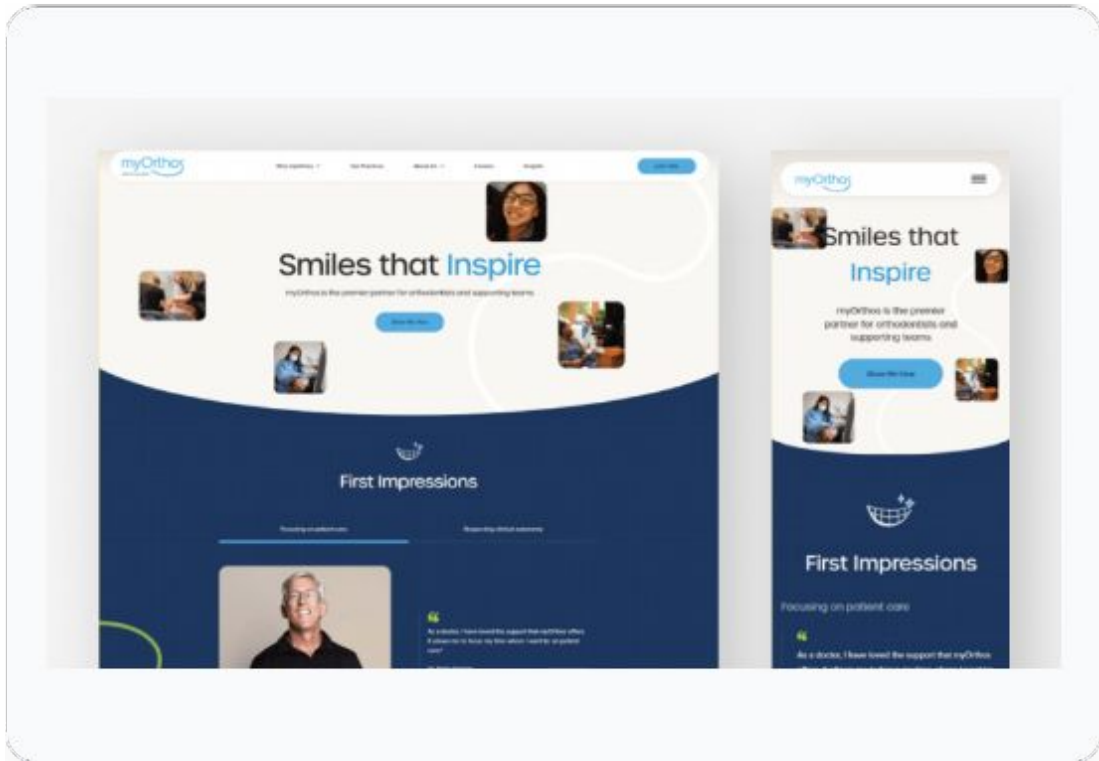
1. Apa fungsi dari klausa modifier sakti `.stop` pada event handler?
2. Jelaskan kapan waktu strategis menggunakan `v-show` ketimbang `v-if`?

## Kunci Verifikasi

1. Bertugas menghentikan rambatan tak diundang (event bubbling) kepada elemen tetangga atasnya.
2. Cocok bila visibilitas blok elemen antarmuka tersebut berpotensi bolak-balik berulang dalam frekuensi teramat sering (toggle).



# Mental Model Komprehensif



## Paradigma Konstruksi Vue

Ketika membangun sebuah modul berfitur masif, selalu lontarkan rumusan tanya jawab strategis berikut di kepala Anda:

User melakukan manuver apa? (**Event**). Apakah mutasi merubah nilai data sistem? (**State**). Apakah blok desain butuh hadir sesaat? (**Conditional**). Berupa kelompok tabel berjejer? (**List**). Ataukah sekadar ladang input data baku? (**Form Binding**).

# Ada pertanyaan?

Anda telah merajut kepingan interaksi kokoh sebagai pengembang  
antarmuka Vue masa depan